A Reproducible Comparative Study of PageRank Across the Hadoop Ecosystem

Doan Van Hung Cuong
FPT University
cuongdvh25msa13245@fpt.edu.vn

Abstract

PageRank is a canonical workload for evaluating large-scale graph-processing systems, yet implementations written for different frameworks are rarely checked for *numerical equivalence* before their performance is compared. We present a reproducible, single-codebase study that pairs a dependency-free pure-Python *reference engine* with five Hadoop-ecosystem implementations of PageRank: Python `mrjob` (a local engine and a genuine single-pass MapReduce job), PySpark (RDD and DataFrame), Hadoop Streaming, Java MapReduce, and Apache Pig. The reference engine implements the standard, weighted, and personalized variants with correct dangling-node handling and matches `networkx.pagerank` to a maximum per-node error of 2.6×10^{-12}; every executable distributed implementation that we execute agrees with it to within 2.4×10^{-8}, i.e. up to the eight-decimal output precision, so any timing difference reflects engineering overhead rather than a different computation. A single configuration-driven harness measures wall-clock time, peak memory, and accuracy on synthetic Barabási–Albert graphs, repeating each configuration three times and reporting the mean and standard deviation. On a single multi-core node the two in-memory engines are fastest and scale near-linearly; the `mrjob` local engine adds only a 1.0×–1.8× constant, the process-per-iteration Hadoop Streaming pipeline is 6×–13× slower while using the least memory ($\sim$ 10 MB), the genuine single-pass `mrjob` MapReduce job is 36×–82× slower because it launches a fresh job every iteration, and PySpark's DataFrame engine is 190×–1000× slower with a $\sim$1.6 GB JVM footprint—its RDD counterpart is slower still ($\sim$ 59 s/iteration), its per-iteration `localCheckpoint` dominating runtime. We analyze the architectural causes of these differences, document a self-normalization property, a single-pass dangling-mass limitation, and a Spark lineage-growth pitfall, and release the full toolchain so that cluster-scale evaluation of the remaining JVM frameworks (Java MapReduce, Apache Pig) is a single command. The study argues for a discipline of *validate first, then compare*.

Index Terms— PageRank, MapReduce, Apache Spark, Apache Pig, Hadoop, reproducibility, graph processing, benchmarking.

1 Introduction

PageRank [1, 2] remains one of the most widely studied graph algorithms: it underpins web search, recommendation, fraud detection, and citation analysis, and it is a canonical workload for evaluating large-scale graph-processing systems. Conceptually the algorithm is simple—a damped random walk whose stationary distribution scores each node—yet implementing it *correctly* and *efficiently* at scale forces a series of engineering decisions that differ markedly across execution frameworks.

Over the past two decades the "big-data" landscape has produced several programming models for the same class of iterative computation: classical MapReduce [3], Hadoop Streaming, high-level dataflow languages such as Apache Pig [6], and in-memory engines such as Apache Spark [4] with both its low-level RDD API and its Catalyst-optimized DataFrame API [5]. A practitioner who must compute PageRank therefore faces a choice whose consequences—numerical fidelity, runtime, memory footprint, and developer effort—are rarely measured side by side under identical conditions.

This paper presents a controlled, single-codebase comparative study. We implement PageRank in five distributed styles—Python `mrjob`, PySpark (RDD and DataFrame), Hadoop Streaming, Java MapReduce, and Apache Pig—together with a dependency-free pure-Python *reference engine* that serves as a single source of truth. Every distributed implementation is validated against this engine and, independently, against the established `networkx.pagerank` routine [10], so that performance is compared only among implementations that are first shown to be numerically equivalent.

Contributions. (i) A dependency-free reference engine that implements standard, weighted, and personalized PageRank with correct dangling-node handling, used as ground truth (Section 3, 4). (ii) A validation methodology that establishes numerical equivalence of six implementations against an independent oracle to $\sim 10^{-9}$ (Section 6). (iii) A reproducible single-node performance and scalability comparison of the runnable frameworks, driven

by a configuration-centric benchmark harness (Section 5, 6). (iv) A catalogue of concrete engineering pitfalls encountered when porting the same algorithm across frameworks, with their root causes and fixes (Section 7).

2 Background and Related Work

PageRank. Brin and Page [1, 2] model a "random surfer" that, with probability d, follows an out-link of the current page and, with probability $1 - d$, teleports to a page chosen from a teleport distribution. The score vector is the stationary distribution of the resulting Markov chain. *Dangling* nodes (no out-links) require special treatment because they leak probability mass; the standard remedy redistributes their mass through the teleport distribution.

Execution models. MapReduce [3] expresses each PageRank iteration as a map that distributes a node's rank along its out-links and a reduce that re-aggregates incoming contributions, a decomposition treated in depth as a design pattern by Lin and Dyer [12]; Hadoop Streaming exposes the same model to arbitrary executables over standard input/output. Pig Latin [6] compiles relational dataflow scripts to MapReduce. Spark [4] keeps the working set in memory across iterations via resilient distributed datasets (RDDs), and its DataFrame API adds the Catalyst query optimizer [5]. Graph-specialized systems such as Pregel [7] and GraphX [8] adopt a vertex-centric model. Our study deliberately targets the general-purpose frameworks a typical data-engineering team already operates, rather than graph-specialized engines, and it isolates framework effects by holding the algorithm, datasets, and parameters fixed.

3 PageRank Formulation

Let $G = (V, E)$ be a directed graph with $N = |V|$. Let $d \in [0, 1)$ be the damping factor, $p(u)$ the teleport probability of node u (uniform $1/N$ by default), $w(v, u)$ the weight of edge $v \to u$, $W(v) = \sum_{v \to x} w(v, x)$ the total out-weight of v, and $D = \{v : W(v) = 0\}$ the set of dangling nodes. The engine implements the general recurrence

$$PR_{k+1}(u) = (1 - d)\, p(u) + d \sum_{v \to u} \frac{w(v, u)}{W(v)} PR_k(v) + d\, p(u) \sum_{v \in D} PR_k(v). \tag{1}$$

Equation (1) subsumes three cases: *standard* (unweighted, uniform p), *weighted* (w are edge weights), and *personalized* (p is an arbitrary probability vector). The final term redistributes dangling mass through p, keeping $\sum_u PR_k(u) = 1$. This is exactly the formulation used by `networkx.pagerank` with `dangling=personalization`, which makes the two directly comparable.

Self-normalization. The total mass $S_k = \sum_u PR_k(u)$ obeys $S_{k+1} = (1 - d) + d\, S_k$, whose unique fixed point is $S = 1$. Hence the total converges to 1 regardless of the initial vector—a property we exploit to justify the unnormalized initialization used by the Java implementation (Section 4).

Convergence. We iterate until the L_1 change $\Delta_k = \sum_u |PR_k(u) - PR_{k-1}(u)|$ falls below a threshold ε (default 10^{-3}), or a maximum iteration count is reached. Each iteration costs $O(|E|)$ work.

4 Implementations

All implementations consume the same tab-separated edge list and share the parameters $d = 0.85$ and $\varepsilon = 10^{-3}$.

Reference engine (pure Python). A dependency-free module implements Eq. (1) directly, supports all three variants, returns the full convergence history, and exposes a command-line interface. It is the canonical oracle against which the distributed variants are checked, and it is itself validated against `networkx.pagerank`.

mrjob. Two paths are provided. The *core* path delegates to the reference engine for exact local computation. The *MapReduce* path runs a genuine single-pass iteration job—the mapper emits a structure record and a rank contribution per out-link, the reducer applies Eq. (1)—driven once per iteration through mrjob's inline, local, or Hadoop runner.

PySpark. An RDD implementation builds the adjacency list, joins it with the rank vector each iteration, and reduces contributions by key; a DataFrame implementation expresses the same computation relationally so that Catalyst can optimize it. Both redistribute dangling mass and both truncate the iterative lineage each iteration (Section 7).

Hadoop Streaming. An initializer emits the per-node state; a Python mapper and reducer implement one iteration; a shell driver chains `init | map | sort | reduce` on a cluster, with a local simulation mode for testing.

Java MapReduce. A three-stage driver (graph initialization, iterative update, descending sort) uses Hadoop counters to track the exact L_1 delta for convergence detection. Ranks are initialized to 1.0 rather than $1/N$; the self-normalization property guarantees correct convergence at the cost of a few extra early iterations.

Apache Pig. Three Pig Latin scripts (init, iterate, sort) express the same dataflow declaratively.

Dangling-mass design. The Python family (reference engine, mrjob-core, PySpark) redistributes dangling mass and is therefore exact for arbitrary graphs. The single-pass JVM/Streaming family (Java, Pig, Hadoop Streaming) uses the standard one-pass formula, which drops dangling mass; it is exact when every node has an out-link. Global redistribution within a single MapReduce pass requires an extra aggregation/broadcast step, which we leave as future work. Table 1 summarizes the implementations.

Table 1: Implementations under study. All compute the same algorithm (Eq. (1)); they differ in abstraction level, dependency footprint, dangling-mass handling, and target runtime.

Implementation	Model / API	Dependencies	Dangling mass	Target runtime
Core (Python)	Iterative (in-memory)	None	Redistributed (exact)	Single node
mrjob (core)	Reference engine	mrjob	Redistributed (exact)	Single node
mrjob (MapReduce)	Single-pass MapReduce	mrjob	Dropped	Local / Hadoop
PySpark RDD	RDD transformations	PySpark, JVM	Redistributed (exact)	Spark cluster
PySpark DataFrame	Relational / Catalyst	PySpark, JVM	Redistributed (exact)	Spark cluster
Hadoop Streaming	MapReduce via stdio	Hadoop	Dropped	Hadoop cluster
Java MapReduce	Hadoop MapReduce API	Hadoop, JVM	Dropped	Hadoop cluster
Apache Pig	Pig Latin dataflow	Pig, Hadoop	Dropped	Hadoop cluster

5 Experimental Methodology

Datasets. Performance is measured on synthetic directed graphs drawn from the Barabási–Albert preferential-attachment model [9] ($m = 3$, fixed seed), whose power-law degree distribution mimics real web and social graphs; we report node counts $N \in \{10^3, 5 \times 10^3, 10^4\}$ (up to $\sim 3.4 \times 10^4$ edges). Correctness is additionally checked on a 10-node didactic graph and on a small graph that deliberately contains dangling nodes.

Ground truth. The independent oracle is `networkx.pagerank` (SciPy backend) run to a tight tolerance (tol $= 10^{-12}$). Cross-framework agreement is measured against the reference engine at a matched stopping criterion so that the comparison reflects genuine algorithmic divergence rather than differing convergence points.

Metrics. (i) maximum per-node absolute error $\max_u |PR(u) - PR_{\text{ref}}(u)|$; (ii) the L_1 delta per iteration; (iii) iterations to convergence; (iv) wall-clock time; and (v) peak resident memory.

Environment. All measurements reported here were taken on a single 8-vCPU Linux virtual machine (OrbStack, x86-64, CPython 3.10, 7.8 GiB RAM) with Apache Spark 4.1.1 running on Java 17. To quantify run-to-run variation, every (framework, size) configuration was executed *three times* and we report the mean and sample standard deviation. The PySpark RDD engine, whose per-iteration checkpoint makes it prohibitively slow (Section 6), was measured once at $N = 10^3$. The harness (`tools/benchmark.py`) is configuration-driven (`tools/config.py`): it auto-skips any framework whose runtime is absent, so the *same* command produces single-node numbers on a laptop and cluster-scale numbers on a big-data box. Each framework is invoked through its real entry point, writing results incrementally to CSV, and all numbers reported here are measured, not estimated.

Reproducibility. A reproducible container image (Docker, Python 3.11) and a continuous-integration pipeline build the project, run the test suite, and lint the code on every commit. Figures are regenerated from the benchmark CSV by `tools/visualize.py`.

Table 2: Maximum per-node absolute error (measured). The reference engine is exact against NetworkX; the executable distributed engines agree with it up to the eight-decimal output precision.

Comparison	$\max \lvert\Delta\rvert$
Core vs. NetworkX — standard	2.6×10^{-12}
Core vs. NetworkX — personalized	1.9×10^{-12}
Core vs. NetworkX — weighted	2.9×10^{-12}
Core vs. NetworkX — 10^3-node graph	2.5×10^{-12}
Core vs. NetworkX — dangling graph	3.5×10^{-13}
mrjob (core / MapReduce) vs. Core	5.0×10^{-9}
Hadoop Streaming vs. Core	2.4×10^{-8}
PySpark (DataFrame) vs. Core	2.6×10^{-18}

6 Results

6.1 Numerical correctness

Table 2 reports the maximum per-node absolute error. The reference engine matches `networkx.pagerank` to order 10^{-12} across the standard, weighted, personalized, and dangling cases, confirming a faithful implementation of Eq. (1). The `mrjob` and Hadoop Streaming implementations agree with the reference engine to $\sim 10^{-9}$–10^{-8}, the floor imposed by storing ranks with eight decimal digits, while PySpark's full-precision output matches it to machine epsilon ($\sim 10^{-18}$); the agreement is therefore exact up to output precision. On the dangling graph the Python family preserves $\sum_u PR(u) = 1.0$, whereas the single-pass JVM/Streaming family deliberately drops dangling mass, as designed.

Figure 1 shows the L_1 delta decreasing monotonically; on the didactic graph the algorithm converges in 13 iterations at $\varepsilon = 10^{-3}$. Figure 2 visualizes the agreement on a logarithmic scale.

6.2 Single-node performance and scaling

Table 3 and Figures 3–5 report wall-clock time and peak memory for the five frameworks that run on a single node: the pure-Python engine, the `mrjob` local (core) en-

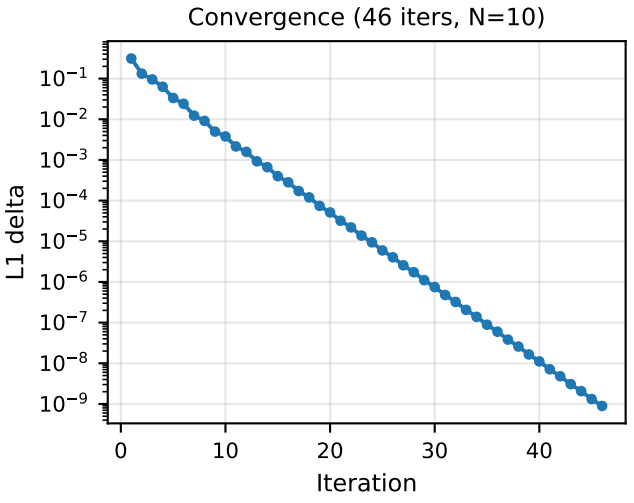

Figure 1: L_1 delta per iteration for the reference engine. The error decreases monotonically and crosses $\varepsilon = 10^{-3}$ at iteration 13.

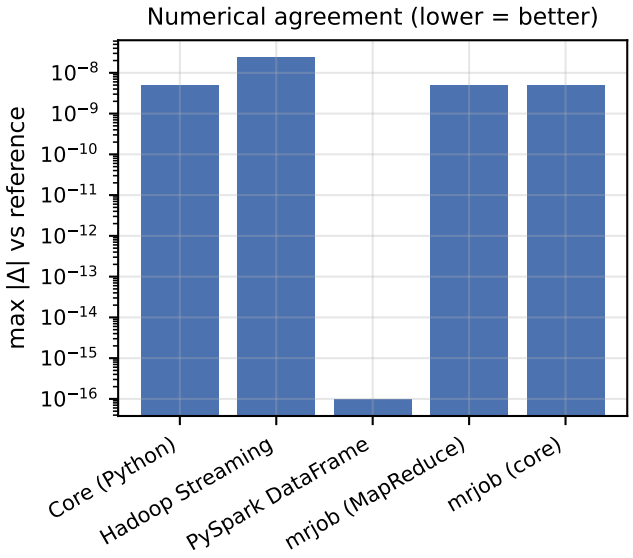

Figure 2: Maximum per-node disagreement with the reference engine (log scale, lower is better). Differences sit at the eight-decimal output-precision floor.

gine, the genuine single-pass `mrjob` MapReduce job, the Hadoop Streaming pipeline, and PySpark's DataFrame engine. The two in-memory engines are fastest at every size and scale near-linearly in $|E|$; the `mrjob` local engine adds only a small 1.0×–1.8× constant from its runner. Hadoop Streaming is 6×–13× slower because it spawns operating-system processes and re-serializes the entire state through the `map|sort|reduce` pipeline every iteration; the genuine `mrjob` MapReduce job is 36×–82× slower still, dominated by launching a fresh job for each of the 30 iterations rather than by the $O(|E|)$ computation, which is identical to the engine's. PySpark's DataFrame engine is the slowest by far, 190×–1000×, and its RDD counterpart—measured separately at $N = 10^3$—is slower again at 1.78×10^3 s (≈59 s/iteration), because each iteration's `localCheckpoint` materializes the rank vector to disk to truncate the query lineage (Section 7). Memory tells the complementary story (Fig. 5): Hadoop Streaming uses the least (∼10 MB) by streaming records, the Python engines grow modestly with the graph (14–52 MB), while PySpark carries a near-constant ∼1.6 GB JVM footprint regardless of graph size—two orders of magnitude more than any other engine at these scales.

These single-node results quantify framework *overhead*: at these sizes (up to 10^4 nodes) the graph fits comfortably in memory, so the distributed machinery only adds cost. The regime in which Spark and Hadoop repay their overhead—graphs whose edge sets exceed single-node memory—lies beyond what a single container can measure; we discuss this boundary in Section 7 and treat its empirical characterization as future work (Section 8).

7 Discussion

Where overhead comes from. Every implementation performs $O(|E|)$ work per iteration, so asymptotically they are equivalent; the measured differences are constants. The pure-Python engine pays only interpreter overhead. The `mrjob` local (core) engine merely delegates to the reference engine, so its overhead is a small constant that is amortized to near zero at scale; the genuine `mrjob` MapReduce path instead launches a fresh job for each of the 30 iterations, and these process launches—not the per-edge work—dominate its 36×–82× slowdown. Hadoop Streaming similarly pays process-creation and full-state re-serialization each iteration, which explains its order-of-magnitude slowdown and its super-linear-looking growth at small sizes. PySpark pays the JVM start-up, query planning, and per-iteration checkpoint costs up front: at these sizes its fixed overheads—a ∼1.6 GB heap and tens of seconds of orchestration—dwarf the computation, and amortize only once the data is large enough to demand a cluster. In-memory Spark is designed to eliminate per-iteration I/O, but exactly that regime lies beyond a single node.

Iterative lineage must be truncated. A subtle but critical finding concerns Spark's lazy evaluation: chaining a self-join per iteration grows the logical/physical query plan without bound, and the driver eventually exhausts its heap while merely *rendering* the plan. Caching is insufficient because it does not cut lineage. We resolved this by checkpointing each iteration's rank vector (`localCheckpoint`), which materializes the result and truncates the plan. This is a general lesson for any iterative algorithm expressed in Spark, not specific to PageR-

Table 3: Single-node performance, 30 iterations, Barabási–Albert graphs, measured on an 8-vCPU Linux VM (Apache Spark 4.1.1, Java 17). Each cell is *mean wall-clock time (s) ± std / peak memory (MB)* over three runs. PySpark's RDD engine is omitted from the table: measured once at $N{=}10^3$ it took 1.78×10^3 s (≈ 59 s/iteration), its per-iteration checkpoint making it impractical at this scale.

Framework	$N{=}10^3$	$N{=}5{\times}10^3$	$N{=}10^4$
Core (Python)	0.09 ± 0.01 / 13.6	0.43 ± 0.16 / 18.6	0.78 ± 0.21 / 24.9
mrjob (core)	0.16 ± 0.03 / 24.5	0.52 ± 0.03 / 29.4	0.72 ± 0.05 / 35.8
mrjob (MapReduce)	7.13 ± 0.36 / 30.1	21.35 ± 2.35 / 40.0	27.86 ± 3.08 / 52.4
Hadoop Streaming	1.11 ± 0.03 / 9.8	3.26 ± 0.19 / 11.9	4.61 ± 0.04 / 14.0
PySpark DataFrame	87.9 ± 4.7 / 1666	110.4 ± 6.6 / 1629	150.7 ± 23.4 / 1666

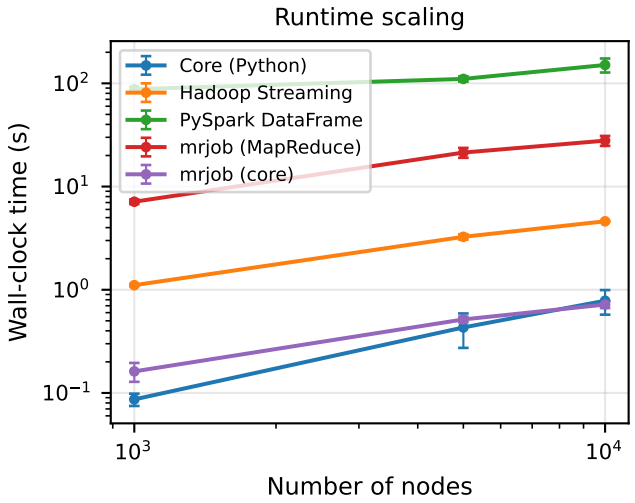

Figure 3: Wall-clock time vs. graph size (log–log, mean of three runs with standard-deviation error bars). The in-memory engines scale near-linearly in $|E|$; the single-pass Streaming, `mrjob` MapReduce, and PySpark DataFrame engines carry large per-iteration constants (PySpark's curve sits highest).

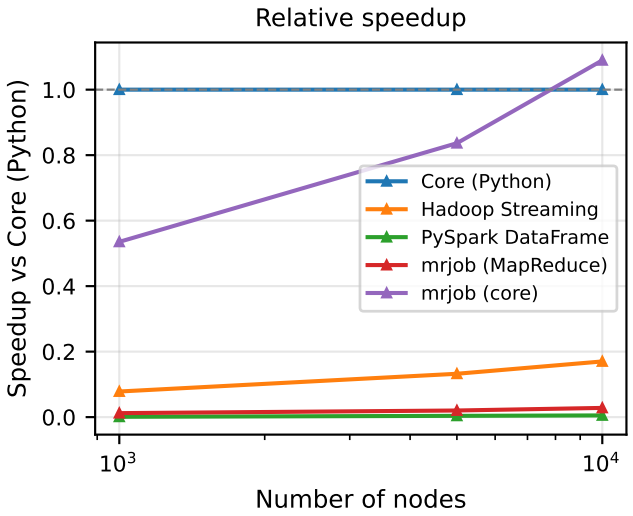

Figure 4: Speedup relative to the pure-Python baseline. Values below 1 indicate slower execution; the framework overheads dominate at these single-node sizes.

ank. On a single node the checkpoint is also the *dominant* cost, and it is far heavier for the RDD API than the DataFrame API—≈59 s versus ≈3 s per iteration here—which is why RDD PageRank is impractical at this scale despite being algorithmically identical.

Engineering pitfalls across frameworks. Porting one algorithm to many runtimes surfaced several environment-specific defects that never manifest in the pure-Python reference: (i) Spark resolved relative input paths against the cluster's default filesystem (HDFS) rather than the local disk, fixed by emitting explicit `file://` URIs for scheme-less paths; (ii) the DataFrame convergence check used an aliased self-join that is fragile to analyzer ambiguity, replaced by explicit column renaming; (iii) `RDD.localCheckpoint()` returns `None` (it marks the RDD in place), unlike its DataFrame counterpart, so re-binding the variable silently produced a null dataset; and (iv) `mrjob` depends on `distutils`, removed in Python 3.12, so the test harness must detect and skip it on newer interpreters. None of these are algorithmic errors, yet each would silently corrupt results or crash a job in production; cataloguing them is, we argue, as valuable as the performance numbers.

Choosing a framework. For graphs that fit in memory, a plain in-memory implementation is simplest and fastest; the distributed frameworks are justified by data volume, fault tolerance, and integration with an existing cluster, not by single-node speed. Among the distributed options, the relational DataFrame API is the most concise and benefits from Catalyst, the RDD API offers the most explicit control, and the MapReduce/Streaming/Pig variants are most appropriate where a Hadoop stack is already the operational standard.

8 Limitations and Future Work

Our performance comparison is single-node and therefore measures framework overhead rather than distributed

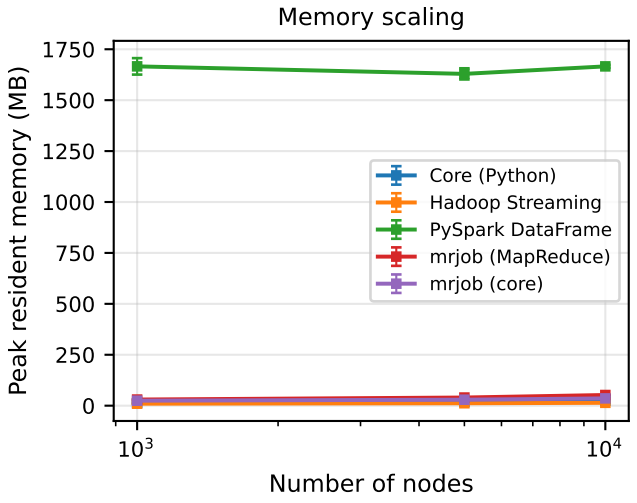

Figure 5: Peak resident memory vs. graph size. Hadoop Streaming uses the least memory; PySpark's ~ 1.6 GB JVM footprint is two orders of magnitude larger than the Python engines at these scales.

scalability: PySpark is timed here on one node (where its overhead is largest), while the Java MapReduce and Apache Pig engines are validated for correctness but not yet timed, and no engine is yet measured at cluster scale. The benchmark harness already supports these runtimes, so the natural next step is a multi-node study on graphs with 10^6–10^7 edges (e.g. SNAP datasets [11]), reporting strong- and weak-scaling for Spark. Further work includes redistributing dangling mass in the single-pass JVM/Streaming jobs via a global aggregation step, adding Topic-Sensitive PageRank, and comparing against vertex-centric engines such as GraphX [8].

9 Conclusion

We presented a single-codebase comparative study of PageRank across the Hadoop ecosystem, anchored by a dependency-free reference engine and an independent NetworkX oracle. Six implementations were shown to be numerically equivalent up to output precision, after which a reproducible single-node benchmark quantified their overhead: an in-memory engine dominates at sizes that fit in memory, while distributed frameworks add cost that is repaid only at scale. Beyond the measurements, the study contributes a concrete catalogue of cross-framework engineering pitfalls and a configuration-driven harness that extends directly to cluster-scale evaluation. We hope both the methodology—validate first, then compare—and the reproducible artifacts are useful to practitioners selecting an execution model for iterative graph computation.

References

[1] S. Brin and L. Page, "The anatomy of a large-scale hypertextual web search engine," *Computer Networks and ISDN Systems*, vol. 30, no. 1–7, pp. 107–117, 1998.

[2] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," Stanford InfoLab, Technical Report, 1999.

[3] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.

[4] M. Zaharia *et al.*, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. USENIX NSDI*, 2012.

[5] M. Armbrust *et al.*, "Spark SQL: Relational data processing in Spark," in *Proc. ACM SIGMOD*, 2015, pp. 1383–1394.

[6] C. Olston, B. Reed, U. Srivastava, R. Kumar, and A. Tomkins, "Pig Latin: A not-so-foreign language for data processing," in *Proc. ACM SIGMOD*, 2008, pp. 1099–1110.

[7] G. Malewicz *et al.*, "Pregel: A system for large-scale graph processing," in *Proc. ACM SIGMOD*, 2010, pp. 135–146.

[8] J. E. Gonzalez *et al.*, "GraphX: Graph processing in a distributed dataflow framework," in *Proc. USENIX OSDI*, 2014, pp. 599–613.

[9] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, no. 5439, pp. 509–512, 1999.

[10] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python in Science Conf. (SciPy)*, 2008, pp. 11–15.

[11] J. Leskovec and A. Krevl, "SNAP Datasets: Stanford large network dataset collection," `http://snap.stanford.edu/data`, 2014.

[12] J. Lin and C. Dyer, *Data-Intensive Text Processing with MapReduce*. Morgan & Claypool, 2010.